

MOHANGRAPHY — Technical Guide & User Manual

NC Mohan · mohangraphy.com · April 2026

This guide covers everything you need to manage your photography website. It is written for day-to-day use — no technical background needed.

Contents

- 1. [System Overview](#)
 - 2. [Day-to-Day Workflow](#)
 - 3. [Photo Tagging — curator.py](#)
 - 4. [Website Navigation & Features](#)
 - 5. [Travel Stories Blog](#)
 - 6. [Storage, Files and Backups](#)
 - 7. [The Deploy Script](#)
 - 8. [Visitor-Facing Features](#)
 - 9. [Email Notifications](#)
 - 10. [Git Commands Reference](#)
 - 11. [Troubleshooting](#)
 - 12. [Quick Reference Card](#)
-

1. System Overview

Mohangraphy is your personal photography portfolio at mohangraphy.com. The entire site is generated by a single Python script on your Mac and published to the internet automatically. There is no CMS, no admin login, and no server to manage.

1.1 How It Works

- 1. Copy photos into the correct folder inside `Photos/` on your Mac
- 2. Run `curator.py` to tag each photo with category, location and caption
- 3. Run `Claude_mohangraphy.py` — it builds the entire website and pushes it live
- 4. Site is live at mohangraphy.com within 30 seconds

1.2 Technology Stack

Component	Technology
Site Generation	Python 3 — <code>Claude_mohangraphy.py</code> (single script)
Hosting	GitHub Pages (free, unlimited)
Database	Supabase — stores likes, visits, subscribers
Email Notifications	GitHub Actions + Resend (<code>send.js</code> in <code>Scripts/</code>)
Analytics	Google Analytics GA4 + Plausible.io

Component	Technology
Fonts	Cormorant Garamond (titles) + Montserrat (body)
Domain	GoDaddy — mohangraphy.com

1.3 All Important Files at a Glance

File / Folder	Purpose
Scripts/Claude_mohangraphy.py	Main script — builds the site (index.html)
Scripts/curator.py	Photo tagging tool — run before deploying new photos
Scripts/blog_editor.py	Blog post editor — create and edit Travel Stories
Scripts/send.js	Email notification script — run by GitHub Actions
Scripts/content.json	Site settings — email, analytics IDs, Supabase credentials
Scripts/photo_metadata.json	All photo tags, categories, locations, captions
Scripts/blog_posts.json	All Travel Stories blog articles
Scripts/list_subscribers.py	Admin tool — exports subscriber list to Excel
.github/workflows/notify.yml	GitHub Action — manual email sender with test/live modes
Photos/	Your original photos — never touched by any script
Thumbs/	Auto-generated 800px thumbnails — safe to delete anytime
Web/	Auto-generated 2048px web copies — safe to delete anytime
index.html	Auto-generated — the entire website. Never edit directly.

 **Never edit** index.html **directly.** It is completely overwritten every time you run Claude_mohangraphy.py.

2. Day-to-Day Workflow

Every time you want to add new photos to the website, follow these steps in order.

2.1 Adding New Photos — Step by Step

- 1. Copy your photos into the correct folder inside Photos/ on your Mac
- 2. Open Terminal (Cmd+Space) → type Terminal → press Enter
- 3. Copy the latest scripts from Downloads:

```
bash
```

```
cp ~/Downloads/Claude_mohangraphy.py /Users/ncm/Pictures/Mohangraphy/Scripts/  
cp ~/Downloads/curator.py /Users/ncm/Pictures/Mohangraphy/Scripts/
```

4. Run the curator to tag your photos:

```
bash  
python3 /Users/ncm/Pictures/Mohangraphy/Scripts/curator.py
```

5. Tag each photo (see Section 3). Click **Stop** when done.

6. Run the deploy script to build and publish the site:

```
bash  
python3 /Users/ncm/Pictures/Mohangraphy/Scripts/Claude_mohangraphy.py
```

7. Done — your photos are live at mohangraphy.com within 30 seconds.

✔ Subscribers receive a branded email when you trigger it manually via GitHub Actions — see Section 9.

2.2 Recommended Folder Structure

```
Photos/  
  Nature/  
    Landscape/ (Megamalai, mountains, sunsets...)  
    Wildlife/  
    Birds/  
    Flora/  
  Architecture/  
  People & Culture/  
  Blog/ (blog-only photos – never shown in Collections)
```

A single photo can belong to multiple categories. A bird photo from Megamalai can be tagged as both **Places/National** AND **Nature/Birds** — it will appear in both sections of the website.

2.3 Moving Photos Between Folders

Action	Result
Move photos into subfolders within same category	✔ Safe — redeploy after
Move photos from one category to another	⚠ Safe, but re-tag via admin editor on site
Rename a folder	✔ Safe — redeploy after
Move photos outside the Photos/ folder	✗ Photos will disappear from site

3. Photo Tagging — curator.py

The curator is a browser-based tool that lets you tag photos quickly. The photo appears on the left and the tagging form on the right — everything in one window.

Important: the curator shows photos that are new (not yet in `photo_metadata.json`) or previously untagged. It will not ask you to re-tag already-tagged photos.

3.1 How to Run

```
bash

python3 /Users/ncm/Pictures/Mohangraphy/Scripts/curator.py
```

A browser window opens automatically at `http://localhost:9797`. When you run it, you will be prompted:

```
Filter mode:
 1 - ALL photos
 2 - UNTAGGED only
 3 - BY DATE
Enter choice:

(If you choose DATE, enter: YYYY-MM-DD)
```

3.2 The Tagging Screen

Field	What to Enter
Photo (left side)	Full photo shown — see exactly what you are tagging
Status badge	Shows NEW PHOTO (green) or ALREADY TAGGED (gold)
Progress counter	Shows "3 / 97" — which photo you are on out of total
Categories	Click buttons to toggle on/off — gold = selected
State / Country	For India: enter State (e.g. Tamil Nadu). For overseas: Country (e.g. Canada)
City	The specific place (e.g. Megamalai or Banff)
Remarks	Optional caption shown as overlay on the photo on the website
Date Added	Defaults to today — or carries forward from previous photo

3.3 Keyboard Shortcuts

Key	Action
<code>Cmd + S</code>	Save current photo and move to next
<code>S</code>	Skip current photo without saving

3.4 Carry-Forward Feature

When tagging a batch from the same location, most fields carry forward automatically:

- **Categories** — carried forward from previous photo
- **State / Country** — carried forward
- **City** — carried forward
- **Date Added** — carried forward

- **Remarks** — always blank (this is unique to each photo)

3.5 Categories Available

Category	Use For
Nature/Landscapes	Mountains, valleys, rivers, scenery, sunsets
Nature/Wildlife	Animals, reptiles, insects
Nature/Birds	All bird photography
Nature/Flora	Flowers, plants, trees
Places/National	Indian locations — requires State + City
Places/International	Overseas locations — requires Country + City
Architecture	Temples, buildings, monuments
People & Culture	Portraits, street scenes, cultural events

✓ Sunsets are included under `Nature/Landscapes` — there is no separate Sunsets category.

3.6 What Gets Saved

Every click of **Save** writes immediately to `photo_metadata.json`. The file is saved photo by photo as you go — there is no separate save step.

4. Website Navigation & Features

4.1 Collections Menu

The main menu has four categories, each accessible from the header or from the homepage tile grid:

- **Landscape** — mountains, sunsets, scenery
- **Flora & Fauna** — wildlife, birds, flowers
- **Architecture** — temples, buildings, monuments
- **People & Culture** — portraits, street, cultural events

Each category has two filter pills at the top: **India** and **Overseas**. Clicking India shows city cards; clicking Overseas shows country cards, which expand into city cards.

4.2 Recently Added Button

The gold pulsing button on the hero section is **always visible**. Its label changes automatically based on whether new photos exist in the last 14 days:

- When new photos exist: *"N photos added recently — view them"*
- When nothing is new: *"No photos added in the past 14 days"*

Clicking the button opens a **Recently Added** gallery showing all photos from the last 14 days, with a **View Slideshow** button at the top.

- ✓ The button is always visible on all screen sizes including mobile — it never hides when no new photos exist.

4.3 Slideshow

A **View Slideshow** button appears at the top of every city gallery and the Recently Added section. Clicking it opens a fullscreen slideshow of all photos in that gallery.

Control	Action
Click anywhere on the photo	Pause / Resume
Arrow buttons or ⬅️ ➡️ keys	Previous / Next photo
␣ key	Pause / Resume
⏏ key	Close slideshow
Swipe left / right on mobile	Previous / Next photo

A gold progress bar at the bottom shows time remaining before the next slide (4.5 seconds per slide). A **PAUSED** label appears at the top when paused.

4.4 Photo Modal

Clicking any photo opens a full-screen detail view:

- Large photo on the left with prev/next arrows
- Right panel with photo title (remarks) and location
- **Like button** — heart turns gold when liked, count stored in Supabase
- **Request Quote button** — opens an email pre-filled with photo details
- **Right-click on photo** — downloads a watermarked JPEG
- **Long-press on mobile** — shows a copyright toast message

4.5 Admin Mode — Right-Click Tag Editor

Right-clicking any photo in a gallery opens a context menu with an **Edit Tags** option. To use it:

1. Right-click any photo
2. Select **Edit tags** from the menu
3. Enter your admin password (set in `content.json`)
4. Edit categories, state, city, and remarks
5. Click **Save** — changes are sent to `patch_tags.py` on your Mac

⚠️ `patch_tags.py` must be running on your Mac for the Save button to update `photo_metadata.json` automatically. If it is not running, the JSON is copied to your clipboard instead.

4.6 Unlocking Admin Mode (Visit Counter Exclusion)

Double-clicking the **MOHANGRAPHY** logo unlocks admin mode. This does two things:

- Right-click Edit Tags becomes available without a password prompt on subsequent uses

- Your browser is permanently marked as owner — your visits are never counted in the visit counter

You only need to do this once per browser / device. The flag is stored in `localStorage`.

Method	How
Automatic (recommended)	Double-click the MOHANGRAPHY logo → enter password once on each device
Manual	Visit <code>https://www.mohangraphy.com?owner=yes</code> on any device

5. Travel Stories Blog

Travel Stories are longer articles about your photo trips — with body text, inline photos, and logistics. They appear under the **Travel Stories** tab in the navigation.

5.1 How to Run the Blog Editor

```
bash
python3 /Users/ncm/Pictures/Mohangraphy/Scripts/blog_editor.py
```

5.2 blog_posts.json Structure

Each post in `blog_posts.json` uses the following fields:

Field	Purpose
<code>id</code>	Unique identifier — used in URLs (e.g. <code>"tadoba-2026"</code>)
<code>title</code>	Article title shown at the top of the post
<code>dates_visited</code>	Trip dates shown below the title
<code>place</code>	Primary location name (e.g. <code>"Tadoba"</code>)
<code>country</code>	Country (leave blank for India)
<code>summary</code>	One-line summary shown on the index list
<code>body</code>	Full article text — paragraphs separated by <code>\n\n</code>
<code>place_tag</code>	Must match city name in <code>photo_metadata.json</code> for the View Photos button to work
<code>collections_links</code>	Category names to link to (optional)

5.3 Body Text Formatting Rules

What You Type	What Appears on the Website
<code>\n\n</code> (two backslash-n)	Paragraph break — starts a new paragraph
<code>\n</code> (one backslash-n)	Line break within the same paragraph

What You Type	What Appears on the Website
<code>## My Heading</code>	Bold sub-heading (start the line with <code>##</code>)
<code>[photo: filename.jpg]</code>	Inserts a photo inline in the article
<code>[photo: filename.jpg Caption]</code>	Inserts a photo with a caption below it

⚠ The `body` field must be one continuous line in JSON — no actual line breaks inside the quotes. Use `\n\n` for paragraph breaks.

5.4 Copyright Protection on Blog Text

All blog text (titles, dates, body text) and the About Me page text are automatically protected against copying:

- If a visitor tries to select and copy text, the clipboard is overwritten with: *"© N C Mohan · mohangraphy.com · All rights reserved"*
- A toast message appears on screen
- Photo right-click protection and watermarked downloads remain in place separately

6. Storage, Files and Backups

6.1 Three Copies of Every Photo

Folder	What It Contains
<code>Photos/</code>	Your originals — full resolution. Never delete these.
<code>Thumbs/</code>	800px thumbnails — auto-generated. Safe to delete anytime.
<code>Web/</code>	2048px web copies — auto-generated. Safe to delete anytime.

✅ `Thumbs/` and `Web/` are completely disposable. Delete them when your drive is full and they will be rebuilt on the next deploy.

6.2 What Gets Published to GitHub

Your original full-resolution photos never leave your Mac. GitHub only receives:

- `index.html` — the entire website structure
- `Scripts/send.js` — email notification script
- `Thumbs/` — 800px thumbnails
- `Web/` — 2048px web copies
- `CNAME` — your domain name

6.3 Freeing Up Hard Drive Space

```
bash
rm -rf /Users/ncm/Pictures/Mohangraphy/Thumbs/*
rm -rf /Users/ncm/Pictures/Mohangraphy/Web/*
```


Then redeploy — thumbnails and web copies are rebuilt automatically. The first deploy after deleting both folders will be slow (10–20 minutes for 500 photos). Subsequent deploys are fast.

6.4 Storage Size Estimates

Item	Approximate Size
Original RAW photo	25–40 MB each
Original JPG photo	8–15 MB each
Web copy (2048px)	1–3 MB each
Thumbnail (800px)	200–400 KB each
1,000 RAW photos (originals)	~30 GB on Mac
1,000 photos on GitHub	~2 GB (web copies only)

 Always keep your originals backed up in two places — an external drive for working files and a second backup (iCloud or another drive). Hard drives do fail.

7. The Deploy Script

7.1 How to Run

```
bash
python3 /Users/ncm/Pictures/Mohangraphy/Scripts/Claude_mohangraphy.py
```

7.2 What Happens When You Run It

Step	Action
1	Reads <code>content.json</code> for site settings and credentials
2	Reads <code>photo_metadata.json</code> for all photo data
3	Reads <code>blog_posts.json</code> for Travel Stories
4	Cleans up dead entries (photos moved or deleted)
5	Generates thumbnails and 2048px web copies for new photos
6	Builds <code>index.html</code> with all CSS, JavaScript, and photo data
7	Commits and pushes to GitHub
8	Site is live at mohangraphy.com within 30 seconds

 Email notifications to subscribers are now sent manually via GitHub Actions — see Section 9.

7.3 What the Output Looks Like

```
=====
MOHANGRAPHY DEPLOY
=====

Step 1 - Checking for deleted photos...
✅ photo_metadata.json is clean

Step 2 - Building site...
OK blog_posts.json loaded - 1 post(s)
Generating thumbnails + 2048px web copies (97 photos)...
Thumbnails ready: 97 | Web copies ready: 97
BUILD COMPLETE

Step 3 - Deploying to GitHub...
✅ Committed: Deploy 2026-04-25 - 12 file(s) updated
✅ Pushed to GitHub successfully!
🌐 Live in ~30 seconds at: https://www.mohangraphy.com
```

7.4 Site Settings — content.json

Setting	Purpose
<code>site.contact_email</code>	Email address on the Get In Touch page
<code>site.ga_measurement_id</code>	Google Analytics ID (starts with <code>G-</code>)
<code>site.supabase_url</code>	Your Supabase project URL
<code>site.supabase_anon_key</code>	Supabase anonymous key for browser features
<code>site.admin_password</code>	Password for right-click Edit Tags and admin mode
<code>site.plausible_domain</code>	Plausible analytics domain

8. Visitor-Facing Features

8.1 Hero Section

- Full-screen slideshow — rotates through Megamalai photos every 3 seconds
- LIGHT · MOMENT · STORY tagline with your name
- EXPLORE button — scrolls down to the Collections grid
- Gold pulsing button (always visible) — shows recently added count, or "no new photos" when nothing is new

8.2 Photo Modal

- Large photo with prev/next arrows or keyboard navigation
- Right panel with title (remarks) and location
- **Like button** — heart turns gold, count stored in Supabase
- **Request Quote button** — opens email pre-filled with photo details
- **Right-click** — downloads a watermarked JPEG

- **Long-press on mobile** — shows copyright toast message

8.3 Subscribe Form

- Name (optional) and email address (required)
- Email validated before submission
- Subscriber saved to Supabase `subscribers` table
- Duplicate emails get a friendly "already subscribed" message

8.4 Visit Counter

- Counts every unique visit to the site
- Shown in the footer as `1,234 visits`
- To exclude your own visits: double-click the MOHANGRAPHY logo → enter password once on each device. Or visit `https://www.mohangraphy.com?owner=yes` on each device separately.

8.5 Copy Protection

What Is Protected	How
Photos	Right-click blocked; long-press on mobile shows copyright message
Photo downloads	Watermarked with MOHANGRAPHY when right-click saved
Blog / Travel Stories text	Cannot be selected or copied
About Me page text	Cannot be selected or copied

If a visitor tries to copy protected text, the clipboard is replaced with "© N C Mohan · mohangraphy.com · All rights reserved" and a toast message appears.

9. Email Notifications

9.1 Overview

Emails are now sent **manually** — you trigger them yourself via the GitHub Actions tab. This gives you full control over when notifications are sent and lets you send a test email first before going live.

- ✓ Emails are no longer sent automatically on every deploy. You must trigger them manually from the GitHub Actions tab on github.com.

9.2 How to Send an Email Notification

1. Go to github.com → your Mohangraphy repository
2. Click the **Actions** tab at the top
3. Click "Notify Subscribers (Manual Only)" in the left sidebar
4. Click "**Run workflow**" on the right
5. Fill in the form fields (see below) and click the green **Run** button

9.3 Workflow Input Fields

Field	Description
What are you sending?	<code>photos</code> for a new photos email, <code>blog</code> for a Travel Story email
Test Only	Set to <code>true</code> to send only to your test email — no real subscribers affected
Test Email	Your personal email for test sends (only used when Test Only is <code>true</code>)
Blog Post Title	Title of the blog post (blog sends only)
Blog Post Place	Location (blog sends only)
Blog Post Summary	One-line summary shown in the email (blog sends only)

- ✓ Always send a test email first (Test Only = `true`) before sending to all subscribers. Check the email looks correct, then re-run with Test Only = `false`.

9.4 Uploading the Notification Files to Git

When you receive a new version of `send.js` or `notify.yml` from Claude, upload them to GitHub with these commands:

```
bash

# Copy files to their correct locations
cp ~/Downloads/send.js ~/Users/ncm/Pictures/Mohangraphy/Scripts/
cp ~/Downloads/notify.yml ~/Users/ncm/Pictures/Mohangraphy/.github/workflows/

# Commit and push to GitHub
cd ~/Users/ncm/Pictures/Mohangraphy
git add Scripts/send.js .github/workflows/notify.yml
git commit -m "Update notification scripts"
git push origin main
```

9.5 The Email Design

- Black background with gold MOHANGRAPHY header
- Personalised greeting — *"Hi [subscriber name],"*
- For photos: *"New photos have just been added to the gallery"* + VIEW NEW PHOTOS button
- For blog: post title, place, summary + READ THE STORY button
- Footer: *"You received this because you subscribed at mohangraphy.com"*

9.6 GitHub Secrets Required

Secret Name	Value
<code>RESEND_API_KEY</code>	API key from resend.com dashboard
<code>SUPABASE_SERVICE_KEY</code>	Supabase service role key (NOT the anon key)

⚠ The email workflow uses `SUPABASE_SERVICE_KEY` (not the anon key). Make sure this secret is set in your GitHub repository under **Settings** → **Secrets** → **Actions**.

9.7 Exporting Your Subscriber List

```
bash

python3 /Users/ncm/Pictures/Mohangraphy/Scripts/list_subscribers.py
```

This creates a timestamped Excel file in your `Scripts/` folder with all subscriber names and emails.

10. Git Commands Reference

These are all the Git commands you need for manual file uploads. The deploy script handles its own Git push automatically — you only need these when uploading files that the script does not manage.

10.1 Uploading the Python Deploy Script

```
bash

cp ~/Downloads/Claude_mohangraphy.py /Users/ncm/Pictures/Mohangraphy/Scripts/
cd /Users/ncm/Pictures/Mohangraphy
git add Scripts/Claude_mohangraphy.py
git commit -m "Update deploy script"
git push origin main
```

10.2 Uploading send.js (Email Script)

```
bash

cp ~/Downloads/send.js /Users/ncm/Pictures/Mohangraphy/Scripts/
cd /Users/ncm/Pictures/Mohangraphy
git add Scripts/send.js
git commit -m "Update email notification script"
git push origin main
```

10.3 Uploading notify.yml (GitHub Action)

```
bash

cp ~/Downloads/notify.yml /Users/ncm/Pictures/Mohangraphy/.github/workflows/
cd /Users/ncm/Pictures/Mohangraphy
git add .github/workflows/notify.yml
git commit -m "Update notification workflow"
git push origin main
```

10.4 Uploading Multiple Files at Once

```
bash
```

```
cd /Users/ncm/Pictures/Mohangraphy
cp ~/Downloads/send.js Scripts/
cp ~/Downloads/notify.yml .github/workflows/
git add Scripts/send.js .github/workflows/notify.yml
git commit -m "Update send.js and notify.yml"
git push origin main
```

10.5 Uploading index.html Manually (Rare)

You should almost never need to do this — the deploy script handles it. Only use this if you have edited `index.html` directly for testing:

```
bash

cd /Users/ncm/Pictures/Mohangraphy
git add index.html
git commit -m "Update index.html directly"
git push origin main
```

⚠ Do not push a manually edited `index.html` and then run the deploy script — the script will overwrite it completely.

10.6 Checking What Git Would Push

```
bash

cd /Users/ncm/Pictures/Mohangraphy
git status      # shows what has changed and is staged
git diff --stat # shows which files changed and by how much
```

11. Troubleshooting

11.1 Common Issues and Fixes

Script gives "SyntaxError: invalid syntax" on line 1

The file was corrupted — a stray character was added when it opened in TextEdit. Fix it:

```
bash

sed -i '' '1s/^[^i]//' /Users/ncm/Pictures/Mohangraphy/Scripts/Claude_mohangrap
python3 /Users/ncm/Pictures/Mohangraphy/Scripts/Claude_mohangraphy.py
```

To prevent this: always right-click downloaded `.py` files and choose **Save Link As** — never open them in TextEdit.

Blog post not showing on the website

The `blog_posts.json` file likely has a formatting error. Check it:

```
bash

python3 -c "import json; json.load(open('Scripts/blog_posts.json'))" && echo OK
```

Photos not appearing in the correct section

Check that the categories in the curator match exactly. The most common mistake is selecting the wrong category — for example putting a bird photo under `Nature/Landscapes` instead of `Nature/Birds`.

International photos not appearing

Make sure you entered the Country name in the **State / Country** field in the curator, and selected `Places/International`. The city should be in the City field.

GitHub Action "Notify Subscribers" fails

Check the following:

- `RESEND_API_KEY` is set in GitHub repository → **Settings** → **Secrets** → **Actions**
 - `SUPABASE_SERVICE_KEY` is set (not the anon key — the service role key)
 - `send.js` is in the `Scripts/` folder (not the root)
-

Visit count includes your own visits

Unlock admin mode: double-click the MOHANGRAPHY logo, enter your password. Your browser is then permanently excluded. Do this on each device you use.

11.2 Safe Commands Reference

Task	Command
Check if metadata JSON is valid	<code>python3 -c "import json; json.load(open('Scripts/photo_metadata.json'))" && echo OK</code>
Check if blog JSON is valid	<code>python3 -c "import json; json.load(open('Scripts/blog_posts.json'))" && echo OK</code>
Delete generated files (safe)	<code>rm -rf Thumbs/* Web/*</code>
See total photos in database	<code>python3 -c "import json; data=json.load(open('Scripts/photo_metadata.json')); print(len(data), 'photos')"</code>
Full redeploy	<code>python3 /Users/ncm/Pictures/Mohangraphy/Scripts/Claude_mohangraphy.py</code>

12. Quick Reference Card

Commands You Use Every Day

Task	Terminal Command
Tag new photos	<code>python3 /Users/ncm/Pictures/Mohangraphy/Scripts/curator.py</code>
Deploy / publish site	<code>python3 /Users/ncm/Pictures/Mohangraphy/Scripts/Claude_mohangraphy.py</code>
Add/edit a blog post	<code>python3 /Users/ncm/Pictures/Mohangraphy/Scripts/blog_editor.py</code>
Export subscriber list	<code>python3 /Users/ncm/Pictures/Mohangraphy/Scripts/list_subscribers.py</code>
Free up disk space	<code>rm -rf /Users/ncm/Pictures/Mohangraphy/Thumbs/*</code> <code>/Users/ncm/Pictures/Mohangraphy/Web/*</code>
Edit site settings	<code>nano /Users/ncm/Pictures/Mohangraphy/Scripts/content.json</code>
Edit blog posts	<code>nano /Users/ncm/Pictures/Mohangraphy/Scripts/blog_posts.json</code>

Files You Must Never Delete

File	Why It Matters
<code>Scripts/Claude_mohangraphy.py</code>	The entire website generation script
<code>Scripts/curator.py</code>	Your tagging tool
<code>Scripts/send.js</code>	The email notification script
<code>Scripts/content.json</code>	Site credentials and settings
<code>Scripts/photo_metadata.json</code>	All your photo tags — years of tagging work
<code>Scripts/blog_posts.json</code>	All your blog articles
<code>Photos/</code>	Your original photos
<code>.github/workflows/notify.yml</code>	Email notification GitHub Action

Files That Are Safe to Delete Anytime

File / Folder	Why It Is Safe
<code>Thumbs/</code>	Rebuilt automatically on next deploy
<code>Web/</code>	Rebuilt automatically on next deploy

The 3 Steps — Every Time You Add Photos

1. `python3 ../curator.py` — tag new photos only
2. `python3 ../Claude_mohangraphy.py` — build and deploy
3. Optional: trigger email notification via GitHub Actions

Git Commands — Manual File Uploads

```
bash

# Always run from inside the Mohangraphy folder first:
cd /Users/ncm/Pictures/Mohangraphy

# Upload Claude_mohangraphy.py
cp ~/Downloads/Claude_mohangraphy.py Scripts/
git add Scripts/Claude_mohangraphy.py && git commit -m "Update script" && git push

# Upload send.js
cp ~/Downloads/send.js Scripts/
git add Scripts/send.js && git commit -m "Update send.js" && git push

# Upload notify.yml
cp ~/Downloads/notify.yml .github/workflows/
git add .github/workflows/notify.yml && git commit -m "Update notify.yml" && git push

# Upload send.js + notify.yml together
cp ~/Downloads/send.js Scripts/ && cp ~/Downloads/notify.yml .github/workflows/
git add Scripts/send.js .github/workflows/notify.yml
git commit -m "Update notification scripts" && git push
```